Proyecto Introducción Sistemas Distribuidos Gestión Inteligente de Tráfico Urbano

Luis Santos
Santiago Hernández Rendón
José D. Medina
Paula Gabriela Losada
Pontificia Universidad Javeriana
Bogotá, Colombia

I. Introducción

El tráfico en las ciudades es un problema que afecta a millones de personas cada día. La congestión genera retrasos, aumenta la contaminación y reduce la calidad de vida de los ciudadanos. Con el aumento constante de vehículos en las ciudades, gestionar el tráfico de forma manual se vuelve cada vez más difícil e ineficiente.

En este proyecto, en su primera versión (Primera Entrega) se propone una solución basada en sistemas distribuidos para la gestión inteligente del tráfico urbano. La idea es simular una ciudad con una cuadrícula de intersecciones (3×5: filas A, B, C y columnas 1 a 5), donde cada punto tiene un semáforo y sensores de tráfico. Estos sensores son de tres tipos: cámaras, espiras inductivas y GPS. La información que generan es procesada por un servicio de analítica que decide cuándo y cómo cambiar los semáforos.

El sistema usa tres nodos (PC1, PC2 y PC3) que se comunican con ZeroMQ mediante patrones asíncronos como PUB/SUB y PUSH/PULL, y el patrón síncrono REQ/REP para las consultas del usuario. También se incluye una réplica de la base de datos en PC2 para que el sistema no se caiga si PC3 falla.

En este documento se explica el diseño del sistema, los modelos, los diagramas, las reglas de funcionamiento, las consultas disponibles, el protocolo de pruebas, las métricas de rendimiento y la implementación inicial de PC1, PC2 y PC3.

II. Objetivos

II-A. Objetivo General

Diseñar e implementar una arquitectura distribuida para la gestión inteligente del tráfico urbano, con sus componentes, reglas de funcionamiento, modelos del sistema, estrategias de evaluación y una implementación inicial.

II-B. Objetivos Específicos

- Analizar el problema del tráfico urbano y plantear una solución distribuida.
- Definir la arquitectura, los componentes y los patrones de comunicación del sistema.
- Describir los modelos del sistema: arquitectónico, de interacción, de fallos y de seguridad.
- Diseñar los diagramas de componentes, despliegue, clases y secuencia.
- Establecer las reglas para los estados de tráfico normal, congestión y priorización.
- Definir qué consultas puede hacer el usuario y qué acciones puede ejecutar sobre el sistema.
- Proponer un protocolo de pruebas funcionales y de desempeño.
- Definir las métricas de rendimiento y cómo medirlas.
- Implementar los servicios de PC1, PC2 y PC3 con persistencia y monitoreo.

III. Marco Teórico

III-A. Sistemas Distribuidos

Un sistema distribuido es un conjunto de computadores independientes que trabajan juntos y se presentan al usuario como un solo sistema [1]. Sus propiedades más importantes son la transparencia (el usuario no nota que hay varios nodos), la escalabilidad y la tolerancia a fallos [2]. En este proyecto estas propiedades se ven en la separación entre PC1, PC2 y PC3, en la réplica de datos y en que cada servicio corre de forma independiente.

III-B. ZeroMQ

ZeroMQ es una librería de mensajería de alto rendimiento que permite comunicar procesos distribuidos sin necesitar un servidor central [3]. Lo que hace interesante a ZeroMQ es que ofrece patrones de comunicación ya listos para usar. En este proyecto se usan tres:

- **PUB/SUB:** el publicador manda mensajes por tópico y los suscriptores reciben solo los que les interesan. Se usa para que PC1 envíe eventos de sensores a PC2.
- **PUSH/PULL:** distribuye tareas entre procesos de forma asíncrona. Se usa internamente en PC2 para pasar comandos de la analítica al control de semáforos, y también para enviar eventos a la base de datos en PC3.
- **REQ/REP:** comunicación síncrona de solicitud y respuesta. Se usa para que el usuario en PC3 pueda consultar el estado del sistema o forzar cambios en los semáforos.

III-C. Broker de Mensajería

El broker es el intermediario que separa a los productores de datos de los consumidores [3]. En este caso, el broker en PC1 recibe los eventos de los sensores y los reenvía a PC2, de modo que ninguno de los dos lados necesita saber nada del otro. Esto hace más fácil escalar el sistema o cambiar componentes sin afectar al resto.

III-D. Tolerancia a Fallos y Réplica

La tolerancia a fallos significa que el sistema sigue funcionando aunque uno de sus nodos falle [2]. Una forma de lograrlo es replicando los datos: mantener copias del estado en varios nodos para que si uno cae, otro pueda tomar su lugar [4]. En este proyecto la base de datos réplica en PC2 se actualiza constantemente, de forma que si PC3 falla, el sistema puede seguir operando sin interrupciones.

IV. Descripción General del Sistema

La ciudad se modela como una cuadrícula de $N \times M$ intersecciones. Cada intersección se identifica con la notación `INT-CK`, donde `C` es la letra de la fila y `K` el número de columna. En la implementación se usa una matriz de 3×5, lo que da 15 intersecciones en total: de `INT-A1` hasta `INT-C5`.

Cada intersección tiene tres tipos de sensores:

- **Espiras inductivas:** cuentan cuántos vehículos pasan en intervalos de 30 segundos (variable C_v).
- **Cámaras:** miden la longitud de la cola (Q) y la velocidad promedio (V_p).
- **GPS:** estiman la velocidad promedio y el nivel de congestión (ALTA, NORMAL o BAJA) en la zona.

El flujo del sistema empieza con los sensores en PC1 que publican eventos via ZeroMQ. La analítica en PC2 los recibe, aplica las reglas y le manda comandos al control de semáforos. Al mismo tiempo, guarda los eventos en la réplica local y los reenvía a la base de datos en PC3. El usuario puede consultar el estado del sistema o mandar comandos directos desde el módulo de monitoreo en PC3.

V. Arquitectura del Sistema

V-A. Componentes del Sistema

- **PC1 — Sensores + Broker ZeroMQ:** Tiene los sensores simulados y expone una API REST en el puerto 3000 para recibir eventos. Los publica por ZeroMQ (PUB) en el puerto 5555 hacia PC2.
- **PC2 — Analítica + Control de Semáforos + BD Réplica:** La analítica (`analitica.js`) se suscribe a los eventos de PC1, aplica las reglas y manda comandos al control de semáforos (`control_semaforos.js`) por PUSH/PULL en el puerto 5560. Escucha comandos del usuario por REP en el puerto 5565 y reenvía eventos a PC3 por el puerto 5570. Mantiene réplica local en `replica_eventos.json` y `replica_semaforos.json`.
- **PC3 — Monitoreo + Base de Datos Principal:** Tiene dos servicios: `bd_principal.js` que recibe todos los eventos por PULL en el puerto 5570 y los guarda en `bd_principal.json`, y `monitoreo.js` que expone una API REST en el puerto 3001 junto con un dashboard web. El monitoreo consulta a PC2 por REQ/REP y permite al usuario ver el estado de la matriz en tiempo real, hacer consultas históricas y forzar cambios de semáforo.

V-B. Patrones de Comunicación

- **PUB/SUB** (PC1 $\rightarrow$ PC2): sensores publican por tópico (`camara`, `espira_inductiva`, `gps`); analítica se suscribe a los tres.
- **PUSH/PULL** (interno PC2): analítica envía comandos al control de semáforos de forma asíncrona.
- **PUSH/PULL** (PC2 $\rightarrow$ PC3): analítica reenvía eventos a la BD principal y los persiste en la réplica local.
- **REQ/REP** (PC3 $\leftrightarrow$ PC2): monitoreo manda solicitudes síncronas a la analítica para consultas y comandos directos.

VI. Modelos del Sistema

VI-A. Modelo Arquitectónico

El modelo arquitectónico del sistema se basa en los principios de los sistemas distribuidos, donde múltiples componentes independientes cooperan para lograr un objetivo común [1], [2]. La arquitectura está organizada en tres nodos principales (PC1, PC2 y PC3), cada uno con responsabilidades claramente definidas, lo que permite separación de preocupaciones, escalabilidad y facilidad de mantenimiento [5].

VI-A1. Distribución de Componentes: El sistema se estructura en tres capas principales:

- **PC1 (Generación de datos)**: Encargado de la recepción y validación de eventos de sensores a través de una API REST. Este nodo publica los eventos utilizando ZeroMQ mediante un socket tipo Publisher en el puerto 5555, permitiendo la difusión de datos hacia los consumidores sin dependencia directa.
- **PC2 (Procesamiento y control)**: Constituye el núcleo del sistema. Este nodo se suscribe a los eventos generados por PC1 mediante un socket Subscriber, procesa los datos utilizando reglas de tráfico y gestiona de forma activa los ciclos de los semáforos. Además, se encarga de:
  - Enviar comandos al servicio de control de semáforos mediante comunicación PUSH/PULL.
  - Mantener el estado interno de cada intersección (variables Q, Vp, Cv).
  - Ejecutar ciclos asíncronos independientes por intersección.
  - Almacenar eventos en una réplica local.
  - Atender solicitudes del usuario mediante comunicación REQ/REP.
- **PC3 (Persistencia y monitoreo)**: Responsable del almacenamiento permanente y la interacción con el usuario. Este nodo recibe eventos desde PC2 y los persiste en la base de datos principal, además de proporcionar una interfaz web para consultas y control del sistema.

VI-A2. Desacoplamiento y Comunicación: El sistema implementa un alto nivel de desacoplamiento mediante el uso de ZeroMQ [3]. En particular:

- El patrón **PUB/SUB** permite que los sensores publiquen eventos sin conocer a los consumidores, facilitando la escalabilidad.
- El patrón **PUSH/PULL** se utiliza para distribuir tareas hacia el control de semáforos y la base de datos, evitando bloqueos en el flujo principal.
- El patrón **REQ/REP** se emplea únicamente para la interacción con el usuario, donde se requiere una respuesta inmediata.

Esta combinación de patrones permite separar completamente la generación, procesamiento y consumo de datos.

VI-A3. Gestión de Estado y Procesamiento Distribuido: El servicio de analítica en PC2 mantiene estructuras de datos en memoria que representan el estado del sistema, como el estado de sensores por intersección y el estado actual de los semáforos. A partir de estos datos, se ejecuta una lógica de evaluación que determina condiciones de tráfico y ajusta dinámicamente los ciclos de los semáforos.

Cada intersección es gestionada mediante un ciclo independiente, lo que permite un comportamiento concurrente y escalable del sistema. Estos ciclos pueden reiniciarse dinámicamente cuando cambian las condiciones del tráfico o cuando se reciben comandos del usuario.

VI-A4. Control Distribuido de Semáforos: El servicio de control de semáforos recibe comandos desde la analítica y actualiza el estado de cada intersección en una matriz que representa la ciudad. Este servicio actúa como un ejecutor pasivo, aplicando únicamente los cambios recibidos, mientras que la lógica de decisión reside completamente en el servicio de analítica.

Esta separación permite desacoplar la toma de decisiones de la ejecución, facilitando la extensibilidad del sistema.

VI-A5. Persistencia y Tolerancia a Fallos: El sistema implementa un esquema de persistencia dual:

- **Base de datos principal en PC3**, donde se almacenan todos los eventos del sistema.
- **Réplica local en PC2**, que actúa como mecanismo de respaldo en caso de fallos en la base de datos principal.

Este enfoque permite garantizar la disponibilidad de los datos y la continuidad del sistema ante fallos parciales [2], [4].

VI-A6. Flujo General del Sistema: El flujo arquitectónico sigue el siguiente patrón:

1. Los sensores generan eventos y los publican en PC1.
2. PC2 recibe los eventos, actualiza el estado y evalúa las reglas.
3. Se generan comandos hacia el control de semáforos.
4. Los eventos son enviados a la base de datos en PC3.
5. El usuario consulta o interactúa con el sistema a través de PC3.

Este flujo refleja una arquitectura orientada a eventos, donde cada componente actúa de manera autónoma y reactiva.

En conjunto, el sistema presenta una arquitectura distribuida, desacoplada y orientada a eventos, que combina comunicación síncrona y asíncrona, procesamiento concurrente y mecanismos de tolerancia a fallos.

VI-B. Modelo de Interacción

El modelo de interacción del sistema describe de manera detallada el flujo de comunicación entre los distintos componentes distribuidos, evidenciando tanto las interacciones síncronas como asíncronas que permiten el funcionamiento del sistema.

Como se muestra en el diagrama de secuencia (Sección VII-B), el proceso inicia en el nodo PC1, donde los sensores de tráflco generan eventos de forma periódica. Estos eventos son enviados al broker de mensajería utilizando el patrón **PUB/SUB**, lo que permite una comunicación asíncrona y desacoplada entre los productores de datos y los consumidores. El broker recibe estos eventos y los reenvía al servicio de analítica en el nodo PC2.

Una vez que el servicio de analítica recibe los eventos, inicia su procesamiento, actualizando el estado de cada intersección y evaluando las reglas de tráfico. Como resultado de este análisis, se generan comandos que son enviados de forma asíncrona al módulo de control de semáforos, permitiendo ajustar el estado de las luces según las condiciones detectadas. Esta comunicación se realiza sin bloquear el flujo principal del sistema.

De manera simultánea, el servicio de analítica envía los datos procesados hacia la base de datos mediante el patrón **PUSH/PULL**. Este mecanismo permite desacoplar el procesamiento del almacenamiento, garantizando que los eventos puedan ser persistidos sin afectar el rendimiento del sistema.

El módulo de control de semáforos, al recibir los comandos, actualiza el estado de las intersecciones y ejecuta los cambios correspondientes (por ejemplo, transición de rojo a verde), manteniendo una representación actualizada de la red de semáforos.

Por otro lado, el usuario interactúa con el sistema a través del módulo de monitoreo en el nodo PC3. Esta interacción se realiza mediante el patrón **REQ/REP**, donde el usuario envía solicitudes de información o comandos al servicio de analítica. En caso de ser necesario, el servicio de analítica consulta la base de datos para obtener información histórica o el estado actual del sistema, y posteriormente responde al usuario con los datos solicitados.

Adicionalmente, el sistema permite la ejecución de comandos directos desde el usuario, como la priorización de una intersección específica. Estos comandos son procesados por el servicio de analítica y enviados al módulo de control de semáforos, integrándose así al flujo normal de operación.

En conjunto, el modelo de interacción muestra un flujo continuo de información que inicia con la generación de eventos, continúa con su procesamiento y almacenamiento, y finaliza con la interacción del usuario, garantizando una operación eficiente, desacoplada y adaptable a las condiciones del tráfico en tiempo real. El modelo de interacción con manejo de errores

incorpora diferentes escenarios de fallo que pueden presentarse durante la ejecución del sistema. Estos incluyen errores en los datos de entrada, fallos en la comunicación entre componentes y fallos en servicios críticos como la base de datos.

El sistema está diseñado para manejar estos errores de forma resiliente, utilizando estrategias como el descarte de eventos inválidos, el uso de valores por defecto, la persistencia en réplicas locales y la validación de comandos del usuario. Esto garantiza la continuidad del sistema incluso ante fallos parciales.

VI-C. Modelo de Fallos

El sistema implementa un modelo de fallos basado en la detección, manejo y recuperación ante errores en cada uno de los componentes distribuidos. Este modelo permite garantizar la continuidad del sistema incluso ante fallos parciales.

VI-C1. Fallos en la Generación de Datos (PC1): En el nodo PC1, los sensores pueden generar datos inválidos debido a errores de entrada o inconsistencias en los valores. Para mitigar esto, el sistema implementa validaciones antes de publicar los eventos:

- Verificación de campos obligatorios en cada tipo de sensor.
- Restricción de valores fuera de rango (por ejemplo, velocidad negativa).
- Validación de coherencia entre variables (nivel de congestión vs velocidad).

En caso de fallo, el evento es rechazado y no se publica, evitando la propagación del error al resto del sistema.

VI-C2. Fallos en la Comunicación (PC1 $\rightarrow$ PC2): El sistema utiliza comunicación asíncrona mediante PUB/SUB, lo que implica que si PC2 no está disponible, los mensajes no son almacenados automáticamente.

Para mitigar este problema:

- El sistema no bloquea la ejecución del sensor.
- Los eventos posteriores siguen generándose normalmente.

Esto permite tolerancia a fallos temporales de conexión sin detener el sistema.

VI-C3. Fallos en el Procesamiento de Eventos (PC2): El servicio de analítica implementa mecanismos para detectar errores en los datos recibidos:

- Manejo de errores en el parseo de JSON.
- Descarte de eventos mal formados.
- Verificación de intersecciones válidas dentro de la matriz.

Cuando ocurre un fallo, el evento es ignorado y el sistema continúa operando sin afectar el estado global.

VI-C4. Fallos en la Lógica de Control: Durante el envío de comandos al servicio de semáforos, pueden ocurrir errores como:

- Intersecciones inexistentes.
- Comandos mal formados.

En estos casos, el servicio de control ignora el comando y registra una advertencia, evitando afectar el resto de la matriz de semáforos.

VI-C5. Fallos en la Base de Datos (PC3): El sistema contempla fallos en la persistencia de datos:

- Si la base de datos no está disponible, los eventos no pueden ser almacenados en PC3.
- Como mecanismo de recuperación, PC2 mantiene una réplica local de los eventos.

Esto permite conservar la información y sincronizarla posteriormente cuando el servicio se restablezca.

VI-C6. Fallos en la Interacción con el Usuario: El sistema de monitoreo implementa control de errores en las solicitudes hacia PC2:

- Uso de timeouts en la comunicación REQ/REP.
- Manejo de errores cuando PC2 no responde.

En estos casos, el sistema retorna un error controlado (por ejemplo, código HTTP 503), evitando bloqueos en la interfaz.

VI-C7. Recuperación ante Fallos: El sistema implementa diferentes estrategias de recuperación:

- Uso de valores por defecto cuando faltan datos de sensores.
- Reinicio automático de ciclos de semáforos ante nuevos eventos.
- Persistencia en archivos de réplica para evitar pérdida de información.

Estas estrategias permiten que el sistema continúe operando de manera estable incluso en presencia de fallos parciales.

VI-D. Modelo de Seguridad

El sistema implementa un modelo de seguridad basado en el control de acceso a nivel de red, validación de datos de entrada y aislamiento de servicios mediante la distribución de componentes en diferentes nodos. Este enfoque permite reducir riesgos asociados a datos inválidos, accesos indebidos y fallos en la comunicación entre componentes.

VI-D1. Segmentación de la Red por Puertos: El sistema define canales de comunicación específicos para cada servicio, utilizando puertos dedicados que separan las responsabilidades:

- 5555: Flujo de eventos de sensores (PC1 $\rightarrow$ PC2)
- 5560: Comunicación interna con control de semáforos
- 5565: Interacción con el usuario (consultas y comandos)
- 5570: Envío de datos hacia la base de datos
- 3001: Interfaz web de monitoreo

Esta segmentación reduce el riesgo de interferencia entre servicios y limita el acceso a cada componente.

VI-D2. Protección de la Integridad de Datos: El sistema protege la integridad de los datos mediante validaciones en múltiples niveles:

- Validación en el origen (PC1) antes de publicar eventos.
- Verificación de formato y contenido en PC2.
- Control de consistencia antes de aplicar reglas de tráfico.

Esto evita que datos corruptos o maliciosos afecten el comportamiento del sistema.

VI-D3. Control de Acceso Lógico: El acceso a las funcionalidades del sistema está controlado por tipo de comunicación:

- Solo PC1 puede publicar eventos de sensores.
- Solo PC2 puede enviar comandos a los semáforos.
- El usuario interactúa únicamente a través de PC3.

Este esquema limita las acciones posibles de cada componente, reduciendo el riesgo de uso indebido.

VI-D4. Protección ante Comandos Inválidos: El sistema valida todos los comandos recibidos desde el usuario:

- Verificación del tipo de comando.
- Validación de parámetros como intersección y duración.
- Rechazo de comandos desconocidos.

Esto evita la ejecución de acciones no autorizadas o inconsistentes.

VI-D5. Aislamiento de Fallos como Mecanismo de Seguridad: El diseño distribuido actúa como mecanismo de seguridad:

- Un fallo en un nodo no compromete a los demás.
- Los servicios operan de forma independiente.

Esto limita el impacto de posibles errores o comportamientos inesperados.

VI-D6. Resiliencia del Sistema: El sistema incorpora mecanismos que fortalecen su seguridad operativa:

- Uso de réplicas locales para evitar pérdida de datos.
- Manejo de errores en todas las capas del sistema.
- Continuidad del servicio ante fallos parciales.

En conjunto, estos elementos permiten mantener la estabilidad del sistema frente a condiciones adversas.

VII. Diseño del Sistema

VII-A. Diagrama de Componentes

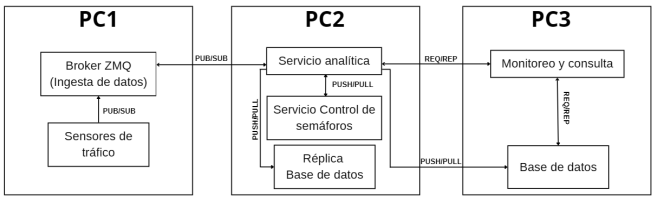

Figura 1. Diagrama de Componentes

VII-B. Diagrama de Secuencia

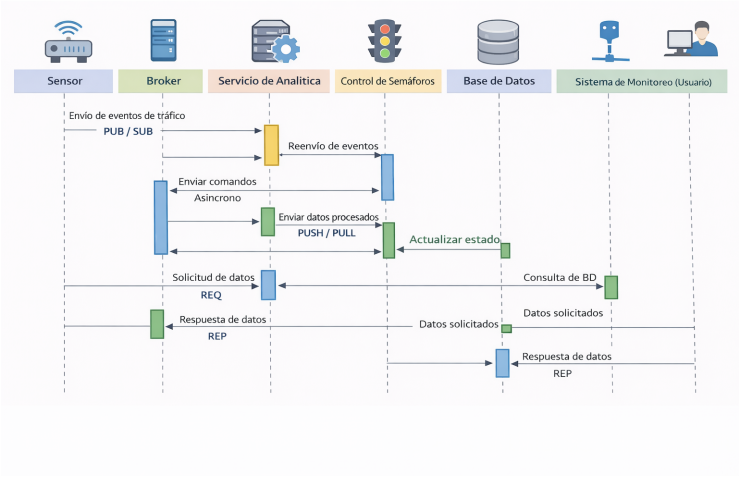

Figura 2. Diagrama de Secuencia

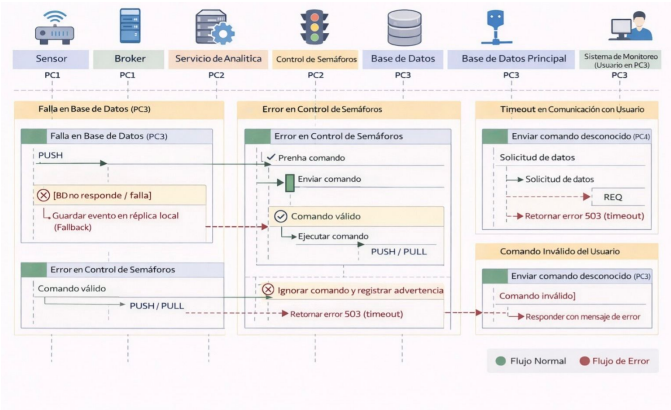

Figura 3. Diagrama de Secuencia - Flujos de error

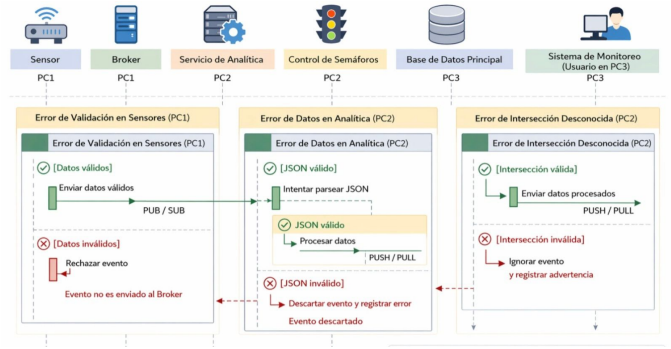

Figura 4. Diagrama de Secuencia - Flujos de error

VII-C. Diagrama de Clases

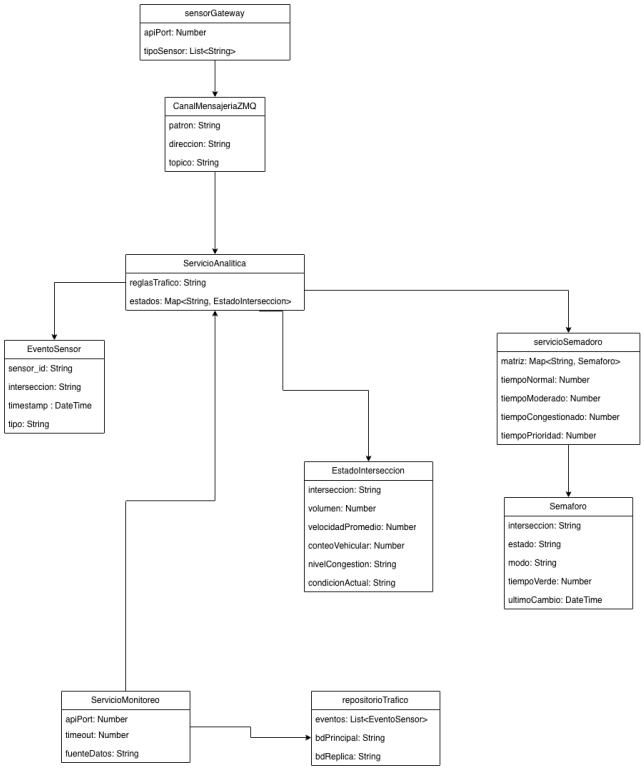

Figura 5. Diagrama de Clases

VII-D. Diagrama de Despliegue

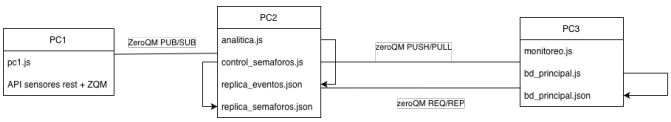

Figura 6. Diagrama de Despliegue

VIII. Definición del Sistema

VIII-A. Inicialización del Sistema

En PC1 se crea un socket PUB en el puerto 5555 y se levanta la API REST en el puerto 3000. Los parámetros de configuración (puerto, dirección ZMQ) están centralizados en el objeto `CONFIG` al inicio de cada archivo para que sea fácil cambiarlos sin tocar la lógica:

```
const CONFIG = {
  API_PORT: 3000,
  ZMQ_PUB_ADDRESS: "tcp://0.0.0.0:5555",
};
const pub = new Publisher();
await pub.bind(CONFIG.ZMQ_PUB_ADDRESS);
```

En PC2, la analítica configura sus cuatro sockets y crea la estructura en memoria que acumula el estado de cada intersección:

```
const CONFIG = {
  PC1_PUB_ADDRESS: "tcp://<IP_PC1>:5555",
  CONTROL_SEMAFOROS_ADDRESS:
      "tcp://127.0.0.1:5560",
  MONITOREO_REP_ADDRESS:
      "tcp://0.0.0.0:5565",
  PC3_BD_ADDRESS: "tcp://<IP_PC3>:5570",
};
const intersectionState = {};
```

Con cada evento recibido se actualiza el estado según el tipo de sensor:

```
if (topic === "camara") {
  intersectionState[interseccion].Q
    = evento.volumen;
  intersectionState[interseccion].Vp
    = evento.velocidad_promedio;
} else if (topic === "espira_inductiva") {
  intersectionState[interseccion].Cv
    = evento.vehiculos_contados;
} else if (topic === "gps") {
  intersectionState[interseccion].Vp
    = evento.velocidad_promedio;
}
```

El control de semáforos inicializa la matriz 3×5 con todos los semáforos en ROJO al arrancar:

```
function inicializarMatriz() {
  const matriz = {};
  for (const fila of CONFIG.MATRIX.ROWS) {
    for (const col of CONFIG.MATRIX.COLS) {
      const id = `INT-${fila}${col}`;
      matriz[id] = {
        estado: "ROJO",
        modo: "NORMAL",
        tiempo_verde: CONFIG.TIEMPOS.NORMAL,
        ultimo_cambio: new Date().toISOString(),
      };
    }
  }
  return matriz;
}
```

PC3 arranca dos procesos de forma independiente. El primero es `bd_principal.js`, que se queda escuchando en el puerto 5570 por PULL y guarda cada registro en `bd_principal.json`. El segundo es `monitoreo.js`, que levanta el servidor web en el puerto 3001 y se conecta a PC2 por REQ para atender las peticiones del usuario:

```
const CONFIG = {
  API_PORT: 3001,
  PC2_ANALITICA_ADDRESS:
      "tcp://127.0.0.1:5565",
  BD_FILE: "./bd_principal.json",
  ZMQ_TIMEOUT_MS: 3000,
};
const zmqReq = new Request();
zmqReq.connect(CONFIG.PC2_ANALITICA_ADDRESS);
```

VIII-B. Definición Inicial de Recursos

Los parámetros del sistema se definen en los objetos `CONFIG` de cada archivo. Esto significa que cambiar el tamaño de la matriz, los tiempos de semáforo o las IPs de las máquinas no requiere modificar la lógica del código, solo el bloque de configuración al inicio.

Los recursos principales son:

- **Matriz de intersecciones:** definida por `MATRIX.ROWS` y `MATRIX.COLS` en `control_semaforos.js`. Por

defecto 3×5 (filas A, B, C; columnas 1 a 5), dando 15 intersecciones y 15 semáforos.

- **Sensores:** un sensor de cada tipo (cámara, espira, GPS) por intersección. En la primera entrega los eventos se generan manualmente vía la API REST de PC1; en la segunda entrega se automatizan con variables aleatorias.
- **Tiempos de semáforo:** configurados en `TIEMPOS` dentro de `control_semaforos.js`: NORMAL = 15 s, MODERADO = 20 s, CONGESTION = 30 s, PRIORIDAD = 60 s.
- **IPs de red:** las direcciones de PC1 y PC3 se cambian en el `CONFIG` de `analitica.js` antes de desplegar en máquinas separadas.

VIII-C. Reglas del Sistema

El sistema evalúa el estado del tráfico con tres variables: longitud de cola (Q), velocidad promedio (V_p) y conteo vehicular (C_v).

VIII-C1. Tráfico Normal: Se considera tráfico normal cuando el flujo es fluido:

$$Q < 5 \text{ AND } V_p > 35 \text{ km/h AND } C_v < 20$$

```
if (q < 5 && vp > 35 && cv < 20) {
  return { condicion: "NORMAL",
           tiempo_verde: 15 };
}
```

El semáforo recibe 15 segundos de luz verde.

VIII-C2. Congestión: Hay congestión si al menos una de estas condiciones se cumple:

$$Q \geq 10 \text{ OR } V_p \leq 15 \text{ km/h OR } C_v \geq 30$$

```
if (q >= 10 || vp <= 15 || cv >= 30) {
  return { condicion: "CONGESTION",
           tiempo_verde: 30 };
}
```

El semáforo se pone en verde 30 segundos para descongestionar. En este caso y en PRIORIDAD el control fuerza directamente el estado a VERDE:

```
if (condicion === "CONGESTION"
 || condicion === "PRIORIDAD") {
  nuevoEstado = "VERDE";
}
```

VIII-C3. Tráfico Moderado: Si no es ni normal ni congestión, se asigna modo moderado con 20 segundos de verde:

```
return { condicion: "MODERADO",
         tiempo_verde: 20 };
```

VIII-C4. Priorización: El usuario puede forzar verde en una intersección desde PC3, por ejemplo para el paso de una ambulancia. En el dashboard hay un formulario que pide la intersección y la duración. Al enviarlo, el monitoreo hace un POST a `/api/ambulancia`, que traduce la petición al comando ZMQ correspondiente:

```
{
  "tipo": "FORZAR_VERDE",
  "interseccion": "INT-C5",
  "duracion": 60
}
```

La analítica lo convierte en un comando de control con condición `PRIORIDAD`:

```
const cmd = {
  tipo: "CAMBIO_SEMAFORO",
  interseccion,
  condicion: "PRIORIDAD",
  tiempo_verde: duracion ?? 60,
  timestamp: new Date().toISOString(),
  origen: "USUARIO",
};
```

El semáforo queda en verde el tiempo indicado y vuelve solo a ROJO al terminar. El campo `origen: ÜSUARIO"` queda registrado en la réplica para poder distinguir los cambios manuales de los automáticos.

VIII-D. Consultas del Usuario

El módulo de monitoreo en PC3 ofrece dos vías de consulta: la API REST y el dashboard web. Internamente ambas usan el socket REQ para comunicarse con la analítica en PC2 por el patrón REQ/REP. Se soportan tres comandos:

VIII-D1. Consultas en Tiempo Real: Consulta de una intersección específica (REST: `GET /api/estado/:interseccion`):

```
{ "tipo": "CONSULTA_ESTADO",
  "interseccion": "INT-C5" }
```

Devuelve los valores actuales de Q, V_p, C_v y la condición evaluada.

Consulta de todas las intersecciones (REST: `GET /api/estado`):

```
{ "tipo": "CONSULTA_TODOS" }
```

Devuelve el estado de las 15 intersecciones con su condición actual. El dashboard hace esta consulta cada 2 segundos automáticamente para mantener la matriz actualizada.

VIII-D2. Consultas Históricas: Se acceden vía `GET /api/historico?desde=&hasta=&interseccion=`. El módulo lee directamente `bd_principal.json` y filtra por rango de timestamps e intersección. Si PC3 está caído, esta misma consulta se puede redirigir a `replica_eventos.json` en PC2:

```
let eventos = bd.eventos ?? [];
if (desde) eventos = eventos.filter(
  e => new Date(e.timestamp) >= new Date(desde));
if (hasta) eventos = eventos.filter(
  e => new Date(e.timestamp) <= new Date(hasta));
if (interseccion) eventos = eventos.filter(
  e => e.evento?.interseccion === interseccion);
```

VIII-E. Acciones del Sistema

- **Cambio automático de semáforo:** cuando se detecta una condición de tráfico, se manda el comando `CAMBIO_SEMAFORO` y el semáforo vuelve a ROJO automáticamente con `setTimeout`.

- **Priorización de vía:** el usuario manda `FORZAR_VERDE` desde PC3 (vía dashboard o API) y el origen queda registrado como `ÜSUARIO"` en la réplica. El tiempo de verde se puede configurar en el formulario del dashboard.
- **Persistencia continua:** después de cada cambio se actualizan `replica_semaforos.json` y `replica_eventos.json` en PC2, y cada evento se reenvía a `bd_principal.json` en PC3.
- **Dashboard en tiempo real:** `monitoreo.js` sirve una interfaz web que muestra la matriz 3×5 de semáforos con su estado (ROJO/VERDE), el modo de tráfico y un log de operaciones. También permite ejecutar consultas históricas y forzar verde directamente desde el navegador.

IX. Protocolo de Pruebas

El plan de pruebas tiene como objetivo evaluar de manera sistemática el comportamiento funcional y no funcional del sistema distribuido. La combinación de estas permite validar tanto la lógica del sistema como su comportamiento en escenarios reales.

IX-A. Pruebas Funcionales

Las pruebas funcionales tienen como objetivo verificar que cada uno de los componentes del sistema cumple correctamente con las funcionalidades para las cuales fue diseñado. Estas pruebas se enfocan en validar el comportamiento esperado del sistema frente a entradas específicas, asegurando que las salidas generadas sean correctas.

En el contexto del sistema distribuido de gestión de tráfico, las pruebas funcionales permiten comprobar aspectos como:

- La correcta generación y envío de eventos por parte de los sensores (PC1).
- La adecuada recepción y procesamiento de datos en el servicio de analítica (PC2).
- La correcta aplicación de las reglas de tráfico y el control de semáforos.
- El almacenamiento adecuado de eventos en la base de datos (PC3).
- La respuesta correcta del sistema ante solicitudes del usuario.

Estas pruebas garantizan que el sistema cumple con los requisitos funcionales definidos y que las interacciones entre los componentes producen los resultados esperados.

IX-A1. Pruebas Unitarias: Las pruebas unitarias se realizan aislando cada componente del sistema.

En PC1, se envían solicitudes HTTP manuales (por ejemplo, usando Postman) a los endpoints de sensores. Se verifica que:

- Los eventos válidos sean publicados correctamente.
- Los eventos inválidos sean rechazados mediante validaciones.

En PC2, se inyectan eventos directamente al módulo de analítica para comprobar que las reglas de tráfico (NORMAL, MODERADO, CONGESTIÓN) se aplican correctamente.

En PC3, se verifica que cada evento recibido se almacena correctamente en la base de datos, validando tanto la estructura como la integridad del registro.

Cuadro I
Pruebas Unitarias

Escenario	**Descripción**
Datos válidos de sensores	Envío de eventos correctos desde PC1
Datos inválidos	Envío de eventos con errores de formato o valores
Procesamiento de reglas	Evaluación de condiciones de tráfico
Persistencia de eventos	Almacenamiento en base de datos

Estas pruebas garantizan que cada componente cumple su función de manera independiente, evitando la propagación de errores en el sistema distribuido.

IX-A2. Pruebas de Integración: Se ejecuta el sistema completo en los tres nodos y se generan eventos desde PC1. Se monitorea el flujo de mensajes:

- Se verifica que PC2 recibe los eventos publicados.
- Se valida que los eventos sean reenviados a PC3.
- Se ejecutan consultas desde PC3 y se valida la respuesta.

Para comandos, se envía una solicitud de priorización (ambulancia) desde PC3 y se observa el cambio en el estado del semáforo.

Cuadro II
Pruebas de Integración

Escenario	**Descripción**
Comunicación PC1–PC2	Envío de eventos mediante PUB/-SUB
Comunicación PC2–PC3	Envío de eventos mediante PUSH/-PULL
Consulta usuario	Interacción REQ/REP desde monitoreo
Comandos de control	Ejecución de FORZAR_VERDE

Estas pruebas validan la correcta interoperabilidad entre componentes, asegurando que los patrones de comunicación distribuidos funcionan correctamente.

IX-A3. Pruebas de Sistema: Se simulan condiciones de tráfico mediante el envío controlado de eventos desde PC1:

- Para tráfico normal, se envían valores bajos de Q, Vp y Cv.
- Para congestión, se envían valores altos o críticos.
- Para prioridad, se ejecuta un comando manual desde PC3.

Se verifica el comportamiento del sistema completo, observando cambios en los semáforos y en la base de datos.

Cuadro III
Pruebas de Sistema

Escenario	**Descripción**
Tráfico normal	Flujo bajo de vehículos
Congestión	Alta densidad vehicular
Prioridad	Activación manual por usuario

Permite validar el comportamiento global del sistema bajo condiciones reales.

IX-B. Pruebas No Funcionales

Las pruebas no funcionales tienen como objetivo evaluar atributos de calidad del sistema que no están directamente relacionados con una funcionalidad específica, sino con la forma en que el sistema opera. Estas pruebas permiten analizar características como el rendimiento, la escalabilidad, la seguridad y la tolerancia a fallos.

En este proyecto, las pruebas no funcionales se enfocan principalmente en:

- **Seguridad:** validación de mecanismos de control de datos inválidos y aislamiento entre componentes.
- **Rendimiento:** evaluación del tiempo de respuesta del sistema y la cantidad de eventos procesados bajo diferentes condiciones de carga.
- **Tolerancia a fallos:** verificación de la capacidad del sistema para continuar operando ante errores o fallos parciales.

Estas pruebas permiten determinar si el sistema cumple con los niveles de calidad requeridos y si es capaz de operar de manera eficiente y robusta en diferentes escenarios.

IX-B1. Pruebas de Seguridad: Se envían entradas incorrectas o incompletas al sistema:

- JSON mal formado.
- Comandos inexistentes.
- Parámetros inválidos.

Se verifica que el sistema no ejecute acciones indebidas y responda con errores controlados.

Garantiza la integridad del sistema frente a entradas inválidas.

Cuadro IV
PRUEBAS DE SEGURIDAD

Escenario	Descripción
Datos maliciosos	Envío de información inválida
Comandos incorrectos	Solicitudes no reconocidas
Acceso indebido	Uso incorrecto de endpoints

IX-C. Pruebas de Rendimiento

IX-C1. Escenarios de Carga: Se controla la carga del sistema mediante:

- Variación del número de sensores activos.
- Incremento progresivo de la frecuencia de eventos.
- Generación concurrente de solicitudes desde el usuario.

Cada escenario se ejecuta durante intervalos definidos para observar el comportamiento. Permite evaluar la escalabilidad del sistema y su comportamiento bajo carga.

Cuadro V
ESCENARIOS DE CARGA

Escenario	Descripción
Carga baja	Pocos eventos y baja frecuencia
Carga media	Flujo constante de eventos
Carga alta	Alta frecuencia y concurrencia

IX-C2. Evaluación de Throughput: Se mide la cantidad de eventos procesados en intervalos de tiempo definidos:

- Se registra el número inicial de eventos.
- Se ejecuta la prueba durante un tiempo fijo.
- Se calcula la diferencia de eventos procesados.

Permite identificar la capacidad máxima del sistema.

Cuadro VI
EVALUACIÓN DE THROUGHPUT

Escenario	Descripción
Carga baja	Procesamiento básico
Carga media	Incremento sostenido
Carga alta	Saturación del sistema

IX-C3. Evaluación del Tiempo de Respuesta: Se mide el tiempo desde que el usuario envía un comando hasta que el semáforo cambia de estado:

- Se registra un timestamp al enviar la solicitud.
- Se registra el momento de ejecución del cambio.
- Se calcula la diferencia.

Permite evaluar la latencia del sistema distribuido.

Cuadro VII
TIEMPO DE RESPUESTA

Escenario	Descripción
Solicitud simple	Cambio de semáforo aislado
Carga media	Varias solicitudes simultáneas
Carga alta	Alta concurrencia

IX-D. Pruebas de Fallos

Cuadro VIII
PRUEBAS DE FALLOS

Escenario	Descripción
Datos inválidos	Eventos incorrectos
Fallo de base de datos	Caída de PC3
Comandos incorrectos	Tipos no válidos
Timeout	Falta de respuesta de PC2

Se simulan fallos controlados:

- Se envían datos inválidos.
- Se detiene el servicio de base de datos.
- Se generan comandos incorrectos.
- Se interrumpe la comunicación entre nodos.

Permite validar la tolerancia a fallos y la resiliencia del sistema.

X. MÉTRICAS DE RENDIMIENTO

Para obtener las métricas de desempeño definidas se implementan estrategias de medición directamente sobre los componentes del sistema, utilizando instrumentación en el código y monitoreo de eventos en tiempo real.

X-1. Estrategia para medir la cantidad de eventos almacenados: La cantidad de solicitudes almacenadas en la base de datos se obtiene mediante la instrumentación del servicio de persistencia en PC3.

Cada vez que se recibe un evento desde el servicio de analítica, el sistema incrementa un contador interno antes de almacenar el registro en el archivo `bd_principal.json`. Este contador permite conocer cuántos eventos han sido procesados en un intervalo de tiempo específico.

Adicionalmente, se puede calcular esta métrica directamente analizando el tamaño del arreglo de eventos almacenados en la base de datos, ya que cada evento es persistido como un registro independiente.

Para obtener el valor final:

- Se registra el número total de eventos al inicio.
- Se registra el número total de eventos después de un intervalo de tiempo.
- La diferencia entre ambos valores corresponde a la cantidad de eventos procesados.

Esta estrategia permite medir el **throughput** del sistema, es decir, la cantidad de operaciones completadas por unidad de tiempo.

La medición de la cantidad de eventos almacenados se implementa directamente en el servicio de base de datos principal (PC3), específicamente en el archivo encargado de la persistencia de eventos.

Para ello, se introduce un contador global que se incrementa cada vez que se recibe y almacena un evento proveniente del servicio de analítica.

```
let totalEventos = 0;

for await (const [msgBuf] of pull) {
  let registro;
  try {
    registro = JSON.parse(msgBuf.toString());
  } catch {
    continue;
  }

  guardarEvento(registro);
  totalEventos++;
}
```

Este contador permite conocer en tiempo real cuántos eventos han sido procesados desde que el sistema inició.

Para obtener la métrica en un intervalo específico, se utiliza el siguiente procedimiento:

- Registrar el valor de `totalEventos` en un tiempo inicial t_0.
- Registrar el valor de `totalEventos` en un tiempo final t_1.
- Calcular la diferencia:

$$Eventos = totalEventos(t_1) - totalEventos(t_0)$$

De forma alternativa, se puede obtener esta información directamente desde el archivo de base de datos:

```
const bd = JSON.parse(fs.readFileSync("bd_principal.json")
    );
const total = bd.eventos.length;
```

Esto permite validar los resultados obtenidos mediante el contador interno.

Finalmente, el throughput del sistema se calcula como:

$$Throughput = \frac{Eventos}{t_1 - t_0}$$

X-2. Estrategia para medir el tiempo de respuesta del sistema: El tiempo de respuesta se mide a partir de la interacción del usuario con el sistema, específicamente cuando se solicita una acción que genera un cambio en los semáforos.

Para ello, se utilizan marcas de tiempo (timestamps) en dos puntos clave del sistema:

- **Inicio**: momento en el que el usuario envía una solicitud desde el módulo de monitoreo (PC3).
- **Fin**: momento en el que el servicio de control de semáforos aplica efectivamente el cambio de estado.

La diferencia entre estos dos tiempos permite calcular el tiempo total de respuesta del sistema:

$$Tiempo\ de\ respuesta = t_{fin} - t_{inicio}$$

Este valor incluye:

- Tiempo de transmisión entre nodos.
- Tiempo de procesamiento en el servicio de analítica.
- Tiempo de ejecución en el control de semáforos.

Esta métrica corresponde al concepto de **latencia**, el cual mide el tiempo que tarda el sistema en responder a una solicitud. El tiempo de respuesta se implementa mediante el uso de marcas de tiempo en dos componentes del sistema: el módulo de monitoreo (PC3) y el servicio de control de semáforos (PC2).

En el módulo de monitoreo, se registra el instante en que el usuario envía una solicitud:

```
const inicio = Date.now();

await socketREQ.send(JSON.stringify(comando));
```

Para poder medir correctamente el tiempo total, este timestamp se incluye dentro del mensaje enviado al sistema:

```
comando.timestamp_inicio = inicio;
```

Este valor viaja a través del sistema (PC3 → PC2 → control de semáforos).

En el servicio de control de semáforos, cuando se aplica el cambio de estado, se registra el tiempo final:

```
const fin = Date.now();
const tiempoRespuesta = fin - comando.timestamp_inicio;
```

Este valor puede ser:

De esta forma, se obtiene el tiempo total que tarda el sistema en procesar una solicitud y ejecutar una acción.

X-3. Instrumentación del Sistema: Para obtener estas métricas de manera precisa, se emplean las siguientes técnicas:

- Uso de funciones de tiempo del sistema (por ejemplo, `Date.now()`) para registrar timestamps.
- Contadores internos en los servicios de analítica y base de datos.

- Registro de eventos en logs para validar los valores obtenidos.

Estas estrategias permiten medir el desempeño sin alterar el comportamiento del sistema.

La instrumentación se integra sin modificar la lógica principal del sistema:

- En PC3 (BD): se añade el contador de eventos.
- En PC3 (Monitoreo): se añade el timestamp de inicio.
- En PC2 (Control): se calcula el timestamp final.

Esto permite medir el desempeño sin afectar el comportamiento del sistema ni introducir bloqueos adicionales.

X-4. Obtención de Datos para Análisis: Los valores obtenidos se almacenan en estructuras simples, lo que permite posteriormente:

- Comparar resultados entre diferentes configuraciones del sistema.
- Construir gráficos de desempeño.
- Analizar el impacto del número de sensores y la frecuencia de eventos.

Almacenarlos en archivos en formato **JSON** permite organizar la información de manera estructurada y facilitar su posterior análisis.

Cada registro puede incluir información como:

- Tiempo de respuesta.
- Marca de tiempo de inicio y fin.
- Identificador de la intersección.
- Tipo de evento procesado.

Un ejemplo de estructura de almacenamiento es el siguiente:

```
{
  "timestamp_inicio": 1690000000,
  "timestamp_fin": 1690000012,
  "tiempo_respuesta": 12,
  "interseccion": "INT-A1",
  "tipo": "PRIORIDAD"
}
```

Adicionalmente, los resultados pueden registrarse en consola para monitoreo en tiempo real:

```
console.log(`[METRICA] Tiempo respuesta: ${tiempoRespuesta
    } ms`);
```

Este enfoque permite almacenar y consultar fácilmente los resultados para su análisis posterior, así como generar reportes y visualizaciones.

En conjunto, estas estrategias permiten obtener de forma clara y precisa las métricas de rendimiento del sistema, cumpliendo con los criterios establecidos para la evaluación del desempeño en sistemas distribuidos.

XI. Implementación Inicial

La primera entrega incluye todos los servicios de PC1, PC2 y PC3 funcionando de forma integrada.

XI-A. PC1 — Sensores y Broker

`pc1.js` levanta una API REST con Express en el puerto 3000 y un socket PUB en el 5555. Tiene cuatro endpoints:

- `POST /sensor/camara`: recibe y publica eventos de cámara con `volumen` y `velocidad_promedio`. Valida que la velocidad esté entre 0 y 50 km/h y que el volumen no sea negativo.
- `POST /sensor/espira`: recibe eventos de espira con `vehiculos_contados` en intervalos fijos de 30 s.
- `POST /sensor/gps`: recibe eventos GPS validando que `nivel_congestion` y `velocidad_promedio` sean consistentes entre sí. Por ejemplo, si el nivel es ALTA la velocidad debe ser menor de 10 km/h; si es BAJA debe superar los 40 km/h.
- `GET /health`: verifica que el servicio está corriendo.

Cada endpoint valida los campos y rangos antes de publicar el mensaje. Si falta algún campo requerido o un valor está fuera de rango, responde con HTTP 400 y un mensaje de error descriptivo.

XI-B. PC2 — Analítica

`analitica.js` se suscribe a los tres tópicos de PC1 y mantiene en memoria el estado de cada intersección en el objeto `intersectionState`. Actualiza Q, V_p y C_v con cada evento y evalúa las reglas. Si hay una condición relevante manda un comando `CAMBIO_SEMAFORO` al control de semáforos.

Cuando se recibe un evento de cámara se actualizan Q y V_p; un evento de espira actualiza C_v; y un evento GPS actualiza V_p y el nivel de congestión. De esta manera el estado acumulado siempre refleja la información más reciente de cada tipo de sensor.

En paralelo guarda los eventos en la réplica local y los reenvía a PC3. En un loop separado atiende los comandos directos del usuario por REP, soportando `FORZAR_VERDE`, `CONSULTA_ESTADO` y `CONSULTA_TODOS`.

XI-C. PC2 — Control de Semáforos

`control_semaforos.js` recibe comandos por PULL y los aplica sobre la matriz en memoria. Para CONGESTION y PRIORIDAD fuerza el estado a VERDE; para NORMAL y MODERADO hace toggle. Después de cada cambio a VERDE programa la vuelta a ROJO con `setTimeout` y actualiza `replica_semaforos.json`.

XI-D. PC3 — Base de Datos Principal

`bd_principal.js` escucha en el puerto 5570 por PULL y persiste cada registro recibido en `bd_principal.json`. La estructura del archivo es:

```
{
  "eventos": [
    {
      "topic": "camara",
      "evento": { ... },
      "timestamp": "2026-02-09T15:10:00Z"
    }...
```

El archivo se inicializa automáticamente si no existe al arrancar el servicio. Cada vez que llega un registro se imprime por pantalla el tópico y la intersección correspondiente junto con el contador acumulado de eventos guardados.

XI-E. PC3 — Monitoreo y Consulta

`monitoreo.js` expone una API REST en el puerto 3001 y un dashboard web. Los endpoints disponibles son:

- `GET /api/estado`: consulta el estado de todas las intersecciones a PC2 por REQ y devuelve el JSON con la matriz completa.
- `GET /api/estado/:interseccion`: consulta una intersección específica. Devuelve los valores de sensores, el semáforo actual y la regla evaluada.
- `GET /api/historico`: lee directamente `bd_principal.json` y filtra los eventos por rango de timestamps e intersección.
- `POST /api/ambulancia`: recibe `{interseccion, duracion}` y envía el comando `FORZAR_VERDE` a la analítica. Si PC2 no responde dentro de los 3000 ms configurados, retorna HTTP 503.

El dashboard web muestra la matriz de semáforos actualizada cada 2 segundos, con código de color (rojo o verde) y el modo de tráfico de cada intersección. Incluye también un formulario para forzar verde, un panel de consulta puntual y uno de consulta histórica con filtro por rango de fechas.

XII. Conclusiones

En este proyecto se diseñó una arquitectura distribuida para la gestión inteligente del tráfico urbano, basada en la separación de responsabilidades entre tres nodos principales (PC1, PC2 y PC3). Esta arquitectura permite estructurar el sistema en módulos independientes encargados de la generación, procesamiento y almacenamiento de información, lo cual facilita su comprensión, mantenimiento y posible escalabilidad.

Uno de los principales aportes del diseño es la integración de múltiples patrones de comunicación proporcionados por ZeroMQ, específicamente PUB/SUB, PUSH/PULL y REQ/REP. Estos patrones permiten un alto nivel de desacoplamiento entre los componentes del sistema, logrando que los productores de datos, los servicios de analítica y los consumidores finales operen de manera independiente. Esta decisión arquitectónica sienta las bases para un sistema flexible, extensible y adaptable a diferentes escenarios de tráfico.

Asimismo, se definieron modelos clave del sistema, incluyendo el modelo arquitectónico, de interacción, de fallos y de seguridad. Estos modelos permiten comprender de manera integral el comportamiento del sistema, anticipar posibles errores y establecer mecanismos de recuperación y protección de la información. En particular, la inclusión de una réplica de datos en PC2 constituye una estrategia importante para la tolerancia a fallos, asegurando la continuidad del sistema ante posibles caídas del nodo principal de persistencia.

El diseño también establece un conjunto claro de reglas para la gestión del tráfico, basadas en variables como la longitud de cola, la velocidad promedio y el flujo vehicular. Estas reglas permiten clasificar el estado del tráfico en diferentes condiciones (normal, moderado, congestión y prioridad), proporcionando una base lógica para la toma de decisiones en el control de semáforos.

Adicionalmente, se definieron las consultas que el usuario puede realizar y las acciones que puede ejecutar sobre el sistema, así como un protocolo de pruebas y métricas de rendimiento orientadas a evaluar el comportamiento del sistema en futuras etapas. Esto garantiza que el diseño no solo contempla la funcionalidad, sino también la forma en que el sistema será validado y medido posteriormente.

En conjunto, el proyecto establece una base sólida para la implementación de un sistema distribuido orientado a la gestión de tráfico en tiempo real. La arquitectura propuesta, junto con los modelos, reglas definidos e implementación inicial permite abordar el problema de manera estructurada, dejando preparado el sistema para su implementación completa y evaluación experimental.

Referencias

[1] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Pearson, 2007.

[2] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Addison-Wesley, 2011.

[3] P. Hintjens, *ZeroMQ: Messaging for Many Applications*. O'Reilly Media, 2013.

[4] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Addison-Wesley, 2012.

[5] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.